MODULE 34 · DOMAIN 4 - OPERATING, MONITORING, SECURING

Identity and access

Execution roles, PassRole, least privilege, and Bedrock API keys.

MODULE 34 of 43 DOMAIN 4 - Monitoring WEIGHT 24%

admin@sthithapragna.com

In one training job there are **at least three identities**, and they are not the same one.

The identities in a machine learning workflow

IDENTITY	WHAT IT DOES
Your user or federated role	Calls the API. Needs permission to create the job
The execution role	Assumed by the job. Reads S3, pulls from ECR, writes output
The pipeline execution role	Runs the pipeline and creates its steps. Module 28
The endpoint's execution role	Loads the model, writes logs and capture data
The caller of the endpoint	A separate identity again, invoking the deployed model

ON THE EXAM

Permission to create a training job and permission to read the training data are different permissions, held by different identities.

Resource-based policies

- An identity policy says what a principal may do. A **resource policy** says who may use a resource.
- For ML, the ones that matter are **S3 bucket policies** and **KMS key policies**.
- **A KMS key policy can deny what an IAM policy allows.** Encrypted data needs both to agree.
- That is a common cause of a job failing to read data it appears to have permission for.
- Cross-account access needs both sides, which module 28's structure depends on.

> ON THE EXAM
>
> Encrypted training data has two gates: permission on the bucket, and permission on the key. Either can deny.

Execution roles

- A role that **SageMaker AI assumes** to act on your behalf while a job or endpoint runs.
- It needs access to the **specific buckets, images, keys and resources** that job uses.
- The tempting shortcut is a broad managed policy, which grants far more than any one job needs.
- **One role per workload** is the discipline. One shared role means every job has every permission.
- It is also what CloudTrail records, from module 31. A shared role obscures who acted.

> ON THE EXAM
>
> The execution role is the identity that actually touches your data. It deserves more attention than it usually gets.

SageMaker Role Manager

- Builds execution roles from **personas** and activities, rather than from a blank policy document.
- So a data scientist persona gets a sensible starting role without somebody writing JSON.
- It is a starting point, not a finished answer. Narrow it to your actual resources afterwards.
- Useful for consistency across teams, which matters more than perfection on any one role.
- The alternative in practice is attaching a broad managed policy and never revisiting it.

> ON THE EXAM
>
> The realistic comparison is not Role Manager against a perfect hand-written policy. It is Role Manager against FullAccess.

Model access in Bedrock

- **Model access is requested per model, per Region**, and is separate from IAM permissions.
- So a principal can have **bedrock:InvokeModel** and still fail, because access was never granted.
- Some models require accepting provider terms before access is granted.
- Granting account access is an administrative action. Granting a principal permission is an IAM action.
- Confusing the two is a common and confusing first-day failure. Module 25 raised it for Regions.

> ON THE EXAM
>
> Two separate gates: the account must have access to the model, and the caller must have permission to invoke it. Both, not either.

iam:PassRole

- To give a role to a service, you need permission to **pass** that role: **iam:PassRole**.
- So creating a training job needs **two** permissions: create the job, **and** pass the execution role.
- Missing PassRole is one of the most common and most confusing failures in a new account.
- It exists to stop a low-privileged user handing a high-privileged role to a service.
- Scope it to **specific role ARNs**, not to every role in the account.

> ON THE EXAM
>
> PassRole is a privilege-escalation control. Without it, anyone who can create a job could run code as any role in the account.

The PassedToService condition key

- **iam:PassedToService** restricts which service a role may be passed to.
- So a role can be passable to SageMaker AI and **not** to EC2 or Lambda.
- AWS managed policies use exactly this, with PassRoleToSageMaker and PassRoleToBedrock statements.
- Combined with specific role ARNs, it makes PassRole narrow rather than blanket.
- Without the condition, PassRole on a broad resource is close to granting the role itself.

> ON THE EXAM
>
> This is the pattern AWS uses in its own managed policies, which makes it a reasonable answer when a scenario asks how to constrain PassRole.

Bedrock now has **API keys**. A lot of material still says it does not.

Short-term and long-term API keys

	SHORT-TERM	LONG-TERM
What it is	A pre-signed credential using SigV4	An IAM service-specific credential
Identity	Inherits your IAM principal	Creates an IAM user underneath
Permissions	Exactly the principal's own	AmazonBedrockLimitedAccess attached
Lifetime	Up to 12 hours, or session end	1 day to no expiration
Region	Only where it was generated	Not Region-bound in the same way
AWS advice	Preferred for production	Exploration only

ON THE EXAM

The bottom row is the exam-relevant one, and the row above 'Identity' explains why AWS says it.

Short-term keys in detail

- Generated from an existing IAM principal, through the console or a token generator package.
- **It inherits both the permissions and the expiration** of that principal. It grants nothing extra.
- Valid up to **12 hours or the remaining session**, whichever is shorter.
- **Region-bound.** A key generated in one Region will not work in another.
- Right when a third-party tool needs a bearer token and cannot do SigV4 request signing.

> ON THE EXAM
>
> It creates no new identity and no persistent credential. That is the entire security argument in its favour.

Long-term keys, and what they leave behind

- One click, and you can call Bedrock. That convenience is the whole appeal.
- **It creates an IAM user**, named BedrockAPIKey-something, with a managed policy attached.
- **That user persists whether or not the key is used**, and outlives whoever created it.
- An IAM user may hold **two** such keys, which is how rotation is intended to work.
- Expiry is configurable from one day to **no expiration**, and the default choice matters.

> ON THE EXAM
>
> The IAM user is the part people do not picture. A key that feels like a token is actually a durable identity in your account.

Phantom IAM users

- Long-term keys accumulate as **IAM users nobody remembers creating**, in a predictable naming pattern.
- A security vendor reports these as a **privilege-escalation path**, and found 15 in one account.
- An attacker who can enumerate users and create access keys can **create keys for a phantom user**.
- That grants the full scope of the attached managed policy, **without any Bedrock credential**.
- So audit for them, delete orphaned ones, and restrict who may create long-term keys at all.

> ON THE EXAM
>
> This is the concrete mechanism behind AWS's 'exploration only' guidance. The convenience creates durable identities you did not plan for.

API key or IAM credentials?

IAM CREDENTIALS

The default

- A role on EC2, Lambda or a notebook.
- Temporary credentials from STS.
- No new identity, nothing persistent.
- The strongest option available.

AN API KEY

When a bearer token is required

- A third-party tool that cannot sign SigV4.
- Short-term if it is production.
- Long-term only for exploration.
- Still logged in CloudTrail.

THE TIEBREAKER

If the caller can assume a role, it should. API keys exist for callers that cannot, not as a convenience for ones that can.

Controlling who can create and use them

- **iam:CreateServiceSpecificCredential** controls generation of long-term keys.
- **iam:ServiceSpecificCredentialAgeDays** caps the expiry, so nobody creates a key that never expires.
- **bedrock:CallWithBearerToken** controls **usage** of any API key.
- **bedrock:bearerTokenType** takes SHORT_TERM or LONG_TERM, so you can allow one and deny the other.
- That combination enforces the policy rather than documenting it.

> ON THE EXAM
>
> Allow SHORT_TERM, deny LONG_TERM, and cap the age. Three controls, and the guidance becomes something the account enforces.

Revoking a key

- **Short-term:** invalidate the session used to generate it, or deny CallWithBearerToken on the **role**.
- **Long-term:** reset or delete the service-specific credential, or deny on the **IAM user**.
- **The remediation target differs**, because the identity behind each key type differs.
- **Keys are passed as authorization headers and are not logged**, so CloudTrail shows the calls, not the key.
- A short-term key expires on its own within twelve hours. A long-term one may not.

> ON THE EXAM
>
> The blast radius is the difference. One expires by itself today; the other may persist until somebody notices.

Least privilege in practice

- **Scope to resources:** specific buckets and prefixes, specific model IDs, specific endpoints.
- **A wildcard on bedrock:InvokeModel** permits every model, including expensive ones. Module 16.
- Separate **read** from **write**, and **build** from **deploy**. Module 28's two pipelines.
- Start from a managed policy to get working, then narrow it before production.
- Narrowing later rarely happens unless somebody schedules it. Put it in the plan.

ON THE EXAM

Module 16 made the same point about cost. Restricting model IDs is a security control and a budget control at once.

Attribute-based access control

- **ABAC** grants access by matching **tags** rather than by listing resources.
- A policy saying 'access resources whose Project tag matches mine' scales to any number of projects.
- **aws:RequestTag** forces tags at creation. **aws:ResourceTag** gates access by them.
- It pairs with module 32's cost allocation tags: one tagging discipline, two benefits.
- It fails if tagging is inconsistent, so the enforcement and the access control go together.

> ON THE EXAM
>
> ABAC is how an account with two hundred endpoints avoids a policy listing two hundred ARNs.

IAM Access Analyzer

- Identifies resources shared with **external principals**: buckets, roles, KMS keys.
- **Unused access findings** highlight permissions nobody has exercised, which is where over-grants hide.
- Policy generation can propose a policy from **actual CloudTrail activity**.
- That turns 'narrow this later' into something concrete rather than aspirational.
- Validate policies before deploying. A wildcard typed in a hurry is hard to spot by eye.

> ON THE EXAM
>
> Generating a policy from observed activity is the practical answer to least privilege, because it starts from what was actually used.

Condition keys worth knowing

CONDITION	WHAT IT CONSTRAINS
iam:PassedToService	Which service a role may be passed to
bedrock:bearerTokenType	Whether short-term or long-term keys may be used
iam:ServiceSpecificCredentialAgeDays	Maximum expiry on a long-term key
aws:SourceVpc and aws:SourceVpce	Calls only from your VPC or endpoint. Module 35
aws:RequestTag and aws:ResourceTag	Enforce tagging, which module 32 needed for cost
sagemaker:VpcSubnets and VpcSecurityGroupIds	Force jobs into a specified network. Module 35

ON THE EXAM

The last two rows are how a policy makes a practice mandatory: untagged or unisolated resources simply cannot be created.

Reviewing access

- Permissions accumulate. Nobody removes access when a project ends or a person moves team.
- **Access Analyzer's unused findings** make the review concrete rather than a reading exercise.
- Review execution roles too, not just people. A role outlives the project that created it.
- Delete orphaned roles, users and keys. Each one is a standing risk with no owner.
- Module 34's phantom users are exactly this problem, created by a convenience feature.

> ON THE EXAM
>
> An access review that reads policies finds nothing. One that starts from unused permissions finds the over-grants immediately.

Studio identity

- A Studio **domain** holds **user profiles**, and each profile can have its own execution role.
- **A shared role across profiles obscures CloudTrail**, from module 31's scenario.
- **Per-profile roles** give attribution, and let different users have different data access.
- IAM Identity Center federation maps a workforce identity to a profile.
- The domain's default role is a floor. A profile role can be narrower.

> ON THE EXAM
>
> This is module 31's audit problem, solved at the point where it is cheap: at domain setup, rather than after an incident.

Break-glass access

- There will be an incident where nobody on shift has the permission needed to fix it.
- **Design that access deliberately**: a role that is normally unassumable, with strong controls.
- Require MFA, alarm on its use, and review every use afterwards.
- The alternative is a standing administrator role somebody keeps 'just in case', which is worse.
- CloudTrail on that role should page somebody, not be reviewed quarterly.

> ON THE EXAM
>
> Break-glass is how least privilege survives an incident. Without it, teams grant permanent access under pressure and never remove it.

Cross-account access

- A role in one account can be assumed by a principal in another, if both sides allow it.
- **Two policies must agree:** the trust policy on the role, and the permissions policy on the caller.
- This is how module 28's deploy pipeline reaches a production account from a build account.
- S3 bucket policies and KMS key policies must also permit the cross-account principal.
- A cross-account denial is usually one of those four, and checking all four is faster than guessing.

> ON THE EXAM
>
> Cross-account failures are rarely one missing permission. They are usually two halves of an agreement, with one half missing.

Who may approve a model

- Model approval in the registry is an **IAM-controlled action**, from module 21.
- So 'who may approve a model for production' is a permission, not a convention.
- Separate **register** from **approve**, so the person who built it is not the only one who signs it off.
- And separate **approve** from **deploy**, which is module 28's two-pipeline structure.
- If everyone can approve, the approval step records nothing an auditor would value.

> ON THE EXAM
>
> This is where governance stops being a document. An approval gate that everyone can pass is not a gate.

SCPs and permission boundaries

- A **service control policy** sets the maximum permissions available in an account, organisation-wide.
- So an SCP can forbid creating long-term Bedrock keys **everywhere**, regardless of local policies.
- A **permission boundary** caps what a role can be granted, even by an administrator of that account.
- Both are ceilings. Neither grants anything on its own.
- An SCP denial looks like an ordinary AccessDenied, which makes it confusing to debug locally.

> ON THE EXAM
>
> An SCP is how an organisation makes a security decision once. It is also why a policy that looks correct can still be denied.

STS and assumed roles

- Assuming a role returns **temporary credentials** that expire, with no long-lived secret anywhere.
- That is why a role on a compute resource is the strongest option in this module.
- Session duration is configurable, and shorter is better where the work allows it.
- **Session tags** can carry attributes into the session, which ABAC policies can then match on.
- An access key on a laptop is the opposite: long-lived, portable, and invisible once copied.

> ON THE EXAM
>
> Every good answer in this module reduces to the same thing: a credential that expires without anybody doing anything.

Service-linked roles

- Some services use a **service-linked role**, predefined and managed by the service itself.
- You cannot edit its permissions, which is the point: the service knows what it needs.
- So a missing capability is sometimes a missing service-linked role rather than a policy gap.
- They are created on first use, or explicitly, and deleting one breaks the feature.
- Distinct from your execution role, which you do own and should narrow.

> ON THE EXAM
>
> Worth recognising so you do not spend an hour trying to edit a role that is not yours to edit.

Identity for agents

- An agent with one service role can do anything for anyone, from module 27.
- **Identity delegation** makes it act with the requesting user's permissions, through OAuth or OIDC.
- Then a user who cannot see a record cannot persuade the agent to fetch it.
- AgentCore Gateway also carries a permissions model over which tools an agent may reach.
- And AgentCore Policy enforces limits at run time rather than in an instruction.

> ON THE EXAM
>
> Module 27's scenario 3 in one line: an instruction asks a model to behave, and delegation removes the capability entirely.

The identity setup to ship with

DECISION	WHY IT BELONGS ON DAY ONE
Per-workload execution roles	A shared role cannot be narrowed later without breaking things
Per-user Studio profiles and roles	CloudTrail attribution cannot be reconstructed afterwards
PassRole scoped with PassedToService	Blanket PassRole is close to granting the roles themselves
Deny LONG_TERM bearer tokens	Phantom users are easier to prevent than to find
A tagging standard, enforced	ABAC and cost allocation both depend on it
A break-glass role	Or the first incident creates a permanent one

ON THE EXAM

Four of these six are far harder to retrofit than to set up, which is the argument for doing them before launch.

Secrets

- Database passwords, third-party API keys and tokens do not belong in code or environment variables.
- **Secrets Manager** stores them, rotates them, and controls access with IAM.
- **Parameter Store** handles configuration, and SecureString parameters for simpler cases.
- The execution role gets permission to **read one secret**, not to read all of them.
- A secret committed to a repository is compromised, and rotating it is the only fix.

> ON THE EXAM
>
> Module 35 covers this properly. It appears here because a secret is an identity, and it needs the same scoping discipline.

Designing data access for ML

- **Prefix-level permissions** beat bucket-level. A role rarely needs a whole bucket.
- Training data, model artefacts and captured inference data have **different sensitivities**.
- **Captured payloads inherit the sensitivity of live requests**, from modules 20 and 30.
- Feature Store, the offline store and the online store can carry different access needs.
- Separate buckets, or at least separate prefixes and keys, make the policies writable.

ON THE EXAM

Data capture and invocation logs quietly create new sensitive stores. They need the access design the source data got.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Cannot create a training job	Missing iam:PassRole for the execution role
Job fails in Downloading with no logs	Execution role cannot read the S3 path. Module 31
Policy looks correct and is still denied	An SCP or permission boundary above the account
Cross-account access refused	Trust policy, caller policy, bucket policy or KMS key policy
Audit cannot identify a person	Shared Studio execution role. Modules 31 and 34
Unexplained IAM users named BedrockAPIKey-	Long-term API keys somebody created
An agent fetched data the user cannot see	One service role, no identity delegation. Module 27

ON THE EXAM

Row three is the one that wastes the most time, because the policy you are reading genuinely is correct.

Prefer an identity that **expires by itself** over one somebody has to remember to remove.

> A production service running on ECS needs to call Bedrock. The security team's standard is that no persistent credentials may exist for any workload.
>
> A developer asks whether to generate a long-term Bedrock API key and store it in Secrets Manager.

What should they do?

A. Generate a long-term key, store it in Secrets Manager, and rotate it every 90 days

B. Generate a short-term key and refresh it from the application every 12 hours

C. Generate a long-term key with no expiry, since Secrets Manager protects it

D. Give the ECS task an IAM role with bedrock:InvokeModel, and use no key at all

◆ PAUSE AND DECIDE

What should they do?

A Generate a long-term key, store it in Secrets Manager, and rotate it every 90 days

Creates a persistent IAM user, which the stated standard forbids. Rotation manages a credential that did not need to exist.

B Generate a short-term key and refresh it from the application every 12 hours

Better than a long-term key, and it builds a refresh mechanism to reproduce what a task role already provides automatically.

C Generate a long-term key with no expiry, since Secrets Manager protects it

Storing a credential safely does not stop it being persistent. A key with no expiry is the worst option here.

D Give the ECS task an IAM role with bedrock:InvokeModel, and use no key at all

An ECS task can assume a role and receive temporary credentials automatically. No new identity, nothing persistent, and nothing to rotate.

THE SIGNAL

API keys exist for callers that cannot assume a role. An ECS task can, so the question of which key type never arises.

> A security review of a shared development account finds eleven IAM users named BedrockAPIKey-, several with no expiry set. Nobody recognises most of them, and three appear unused.
>
> The team wants to prevent recurrence without blocking developers from exploring Bedrock.

What should they implement?

A. Delete the users and rely on a written policy that long-term keys are not permitted

B. Deny LONG_TERM via bedrock:bearerTokenType, and cap expiry with the credential age condition

C. Rotate every existing long-term key on a 30-day schedule

D. Remove bedrock:InvokeModel from all developers

◆ PAUSE AND DECIDE

What should they implement?

A. Delete the users and rely on a written policy that long-term keys are not permitted

Cleanup is necessary and a written policy is not a control. The same eleven users reappear over the next quarter.

B. Deny LONG_TERM via bedrock:bearerTokenType, and cap expiry with the credential age condition

Enforces the rule in the account rather than in a document, and short-term keys still let developers explore freely.

C. Rotate every existing long-term key on a 30-day schedule

Maintains credentials that should not exist, and leaves the phantom users in place as escalation targets.

D. Remove bedrock:InvokeModel from all developers

Blocks the exploration the scenario says must continue. It treats the model access as the problem rather than the key type.

THE SIGNAL

The requirement is 'prevent recurrence without blocking exploration'. Only a condition-key control does both at once.

> A data scientist can list and describe training jobs but receives AccessDenied when creating one. The error names their own federated role, and the action is not CreateTrainingJob.
>
> The execution role they are specifying has full access to the relevant bucket and image.

What is missing?

A. iam:PassRole on their role, scoped to the execution role's ARN

B. sagemaker:CreateTrainingJob on the execution role

C. A trust policy allowing the data scientist to assume the execution role

D. S3 read permission on their federated role

◆ PAUSE AND DECIDE

What is missing?

A iam:PassRole on their role, scoped to the execution role's ARN

Creating a job needs two permissions: create the job, and pass the execution role to the service. The error naming an action other than CreateTrainingJob is the tell.

B sagemaker:CreateTrainingJob on the execution role

The execution role is assumed by the job to do work. It does not create the job, so it does not need that permission.

C A trust policy allowing the data scientist to assume the execution role

The user does not assume the execution role. SageMaker AI does, and its trust policy already permits that.

D S3 read permission on their federated role

The execution role reads S3 during the job, and the scenario says it already can. The user's own S3 access is irrelevant here.

THE SIGNAL

The tell is in the error: the denied action is not CreateTrainingJob. PassRole is a separate permission on a separate service.

> During a design review, an engineer states that Bedrock has no API keys and that all access must go through IAM roles. They cite a guide published earlier this year.
>
> The team has a third-party evaluation tool that only supports bearer token authentication and cannot sign requests with SigV4.

How should you respond?

A Agree. Wrap the tool in a proxy that signs requests with SigV4

B Agree, and replace the tool with one that supports IAM roles

C Correct it. Bedrock API keys exist, and a short-term key fits this case

D Correct it, and issue a long-term key with no expiry for simplicity

◆ PAUSE AND DECIDE

How should you respond?

A. Agree. Wrap the tool in a proxy that signs requests with SigV4

Real engineering work to solve a problem the service already solved. It also adds a component to operate and secure.

B. Agree, and replace the tool with one that supports IAM roles

Replaces a working tool on a false premise. The constraint the engineer described no longer exists.

C. Correct it. Bedrock API keys exist, and a short-term key fits this case

A tool needing a bearer token and unable to sign SigV4 is exactly the documented case for a short-term key, which inherits the principal's permissions.

D. Correct it, and issue a long-term key with no expiry for simplicity

Right about keys existing, wrong about which. A key with no expiry creates a permanent IAM user, which AWS advises against.

THE SIGNAL

The engineer's source was correct when written. That is how a stale fact survives: it was true, and nobody re-checked.

What to take from this module

- **Creating a job needs PassRole as well**, ideally with iam:PassedToService.
- **Short-term keys inherit your principal and expire in 12 hours.** Region-bound.
- **Long-term keys create a persistent IAM user.** Exploration only, per AWS.
- **bedrock:bearerTokenType** lets you allow short-term and deny long-term outright.
- **Prefer credentials that expire by themselves** over any you must remember to remove.

> ON THE EXAM
>
> Module 35 continues the security run: VPC mode, endpoints, network isolation and encryption.